

Wham — Smart Contract Construction Specification

Solidity 0.8.28 · Arbitrum · EIP-1167 · EIP-712 · ERC-8004

- Wham — Smart Contract Construction Specification
 - 1. How to use this document
 - 2. Mechanism design
 - 3. Contract inventory
 - 4. Shared types, errors, and the factory
 - 5. WhamCircleFactory
 - 6. WhamCircle — storage and initialization
 - 7. WhamCircle — functions
 - 8. WhamReputation
 - 9. The agent quote and ERC-8004
 - 10. WhamInsurancePool
 - 11. Security model
 - 12. Test matrix
 - 13. Deployment
 - 14. Build order
 - 15. Worked end-to-end scenario

Wham — Smart Contract Construction Specification

Version: 1.0 **Companion to:** Wham — Frontend Construction Specification v1.0 and Wham — .NET Backend Construction Specification v1.0 **Target:** Arbitrum Open House Singapore Online Buildathon (Sep 13 - Oct 4, 2026) **Scope:** Complete contract system — mechanism design, storage layout, every function with preconditions and effects, the settlement math, security model, invariants, test matrix, and deployment. Written so that an agent given only this document can build and ship the system.

1. How to use this document

Sections §3–§9 are per-contract build specifications. Each function is specified as **signature → preconditions → effects → events → reverts**. Build in the order given in §14.

Three corrections to the companion documents are made here. Where they disagree with this document, this document is correct, because these were open questions at the time those were written:

#	Companion doc said	Corrected here	Why
1	Backend computes reputation score, contract records it (Backend §9)	Contract computes the score entirely on-chain; backend projects it	A reputation the backend can rewrite is not a reputation. Everything the score depends on — payment timing, misses, completions — is already on-chain. §7
2	Agent calls <code>setCollateralRequirement()</code> per participant (original brief)	Agent signs an EIP-712 quote the member submits with <code>join()</code>	No agent transaction, no gas cost, no dependency on agent liveness at join time, and the collateral is bounded by contract-enforced floors regardless of what the agent signs. §5.4, §10.2
3	Frontend §8.3 lists <code>claimPayout</code> and <code>withdrawCollateral</code> only	A third withdrawal path, <code>withdrawRewards()</code>, is required	Discount redistribution to waiting members has to be

One new circle parameter is introduced in §2.5 (`payoutCollateralBps`) that does not appear in the companion documents. It closes the exploit flagged as open question #4 in both. It changes the join flow's displayed numbers, so the frontend spec §9.9 and §9.13 need updating — see §2.5 for exactly what changes.

2. Mechanism design

Read this section before writing any code. Every function in §5 is an implementation detail of what follows.

2.1 The instrument

N members each contribute C tokens per round, for N rounds. Each round, one member receives the pot. After N rounds every member has paid $N \times C$ and received one pot. Members who receive early have effectively borrowed; members who receive late have effectively saved.

The contract's job is to make that work between people who do not know each other. It does so with four mechanisms, and every one of them exists to answer the question *"what stops someone taking the pot and walking away?"*

2.2 Mechanism 1 — Collateral

Every member posts a deposit before the circle starts. Missed contributions are taken from it automatically at settlement. The deposit is returned in full when the circle completes.

The deposit size is set **per member** by the underwriting agent (§5.4), bounded by contract-enforced floors and ceilings that hold even if the agent key is compromised (§10.2).

2.3 Mechanism 2 — Discount bidding

Payout order is auctioned rather than fixed or random. Members bid a **discount** — how much of the pot they will forgo to receive it this round. Highest discount wins. The forgone amount is redistributed to the members who have not yet been paid.

This does two things at once. It allocates early payouts to whoever values them most, and it makes early payouts — which carry more default risk, because the recipient still owes many rounds — cost more. The discount is a risk premium paid by the person who creates the risk, to the people exposed to it.

Eligibility: a member who has already been paid out cannot bid. In round r , exactly $N - r$ members are eligible. In the final round exactly one member remains eligible, so no auction is held and the discount is zero.

No bids: if a round closes with no bids, the winner is the eligible member with the lowest `joinIndex` who has not been paid out, at a discount of zero. Deterministic, no randomness, no oracle.

2.4 Mechanism 3 — Insurance pool

A skim of `insuranceSkimBps` is taken from every pot and sent to a pool shared across all circles. When a member defaults and their collateral does not cover the shortfall, the pool makes up the difference so the members still waiting are paid in full.

2.5 Mechanism 4 — Payout-contingent collateral top-up

This is the mechanism the companion documents flagged as unresolved, and it is the most important design decision in the system.

The problem: a member who wins round 0 of an 8-round circle receives roughly $8C$ and still owes $7C$. Their deposit is $1 - 2 \times C$. Nothing meaningful stops them from disappearing. Discount bidding prices this risk but does not collateralise it.

The naive fix — require collateral covering the full remaining obligation — destroys the product. A round-0 winner would need $7C$ posted to receive $8C$, netting C . Nobody would join.

The fix used here: when a member is paid out, the contract raises their required collateral to cover a configured fraction of their remaining obligation, and **funds the top-up from the payout itself**. No new capital is demanded from the member; their payout is smaller and their deposit is larger.

```

remainingObligation = (N - 1 - r) × C
requiredAtPayout    = max(joinCollateral, remainingObligation × payoutCollateralBps / 10000)
topUp               = requiredAtPayout - collateralPosted      (zero if already sufficient)
amountReceived      = netPayout - topUp

```

Worked example, $N = 8$, $C = 50$ USDC, $\text{payoutCollateralBps} = 5000$ (50%), join collateral 100 USDC, skim 1.5%:

Wins round	Remaining owed	Required collateral	Top-up	Gross pot	Net payout (0% discount)	Actually received
0	350.00	175.00	75.00	400.00	394.00	319.00
2	250.00	125.00	25.00	400.00	394.00	369.00
4	150.00	100.00	0.00	400.00	394.00	394.00
7	0.00	100.00	0.00	400.00	394.00	394.00

The top-up is refunded with the rest of the deposit at completion, so it costs the member nothing in the end — it is deferred, not lost. But it converts the highest-risk position in the circle from 1.1% collateralised to 50% collateralised, and it does so without asking anyone for extra money.

Choosing $\text{payoutCollateralBps}$:

Value	Round-0 winner receives (of 394)	Coverage of their remaining obligation	Assessment
0	394.00	29% (join collateral only)	The exploit, unmitigated
2500	381.50	25%	Too weak to matter
5000	319.00	50%	Recommended default
7500	231.50	75%	Strong, but early payouts start losing their appeal
10000	144.00	100%	Fully safe, product is dead

Ship 5000 . Make it a per-circle parameter so it can be tuned, with a hard ceiling of 10000 .

Frontend changes this requires — flag these to whoever owns the frontend: - The bid composer (frontend \$9.13) must show the top-up as a line in the live consequence panel: *“You’d receive 319.00 USDC now (34.00 discount, 41.00 added to your deposit and returned at the end).”* - The `PotBreakdown` component gains a top-up row. - The join flow’s `CommitmentSummary` should mention that winning early increases the deposit, funded from the pot.

Presented correctly this is a **feature**, not a cost: it is the mechanism that makes it safe to let strangers take the pot first, and it is a better story than “we hope they pay.”

2.6 Settlement math

For round r of a circle with N members and contribution C :

```

grossPot      = N × C                                // full pot if everyone pays
skim          = grossPot × insuranceSkimBps / 10000
discountBps   = winning bid, or 0
discountAmount = grossPot × discountBps / 10000
netPayout     = grossPot - skim - discountAmount

waitingCount  = N - r - 1                            // not yet paid, excluding this winner
perWaitingMember = waitingCount > 0
                ? discountAmount / waitingCount
                : 0
redistDust    = discountAmount - (perWaitingMember × waitingCount)

```

`redistDust` (at most `waitingCount - 1` base units) is added to the insurance skim. Never leave dust stranded in the contract; the invariant in §11 depends on every unit being accounted for.

Shortfall handling. `grossPot` assumes every member paid. For each member who did not contribute to round r :

```

owed      = C
available = collateralPosted - collateralSlashed
drawn     = min(owed, available)
collateralSlashed += drawn
shortfall += owed - drawn

```

After the loop, if `shortfall > 0`, the contract requests `shortfall` from the insurance pool. Whatever the pool provides reduces the shortfall. Any residual shortfall reduces `netPayout` **before** it is credited to the winner — the winner cannot be paid money the circle does not hold.

Order of operations in `settleRound` matters and is not negotiable:

1. Determine winner and discount from the bid state
2. Collect: for each member, if not contributed → slash collateral, accrue shortfall
3. Compute grossPot from actual funds held for this round (contributions + slashed collateral)
4. Draw from insurance pool for any residual shortfall
5. Compute skim, discountAmount, netPayout from the actual pot
6. Compute the winner's collateral top-up (\$2.5), deduct from netPayout
7. Credit netPayout to the winner's pending payout balance
8. Credit perWaitingMember to each waiting member's pending rewards balance
9. Transfer skim + dust to the insurance pool
10. Update member and round states, record reputation for on-time/late/missed
11. Advance currentRound, or complete the circle if this was the last round

Steps 7 and 8 **credit balances; they do not transfer**. All outbound token movement is pull-based (\$10.1).

2.7 Round timing

All timestamps derive from two immutable values, so the contract is the single source of truth and the backend's displayed dates are a projection of the same formula:

```

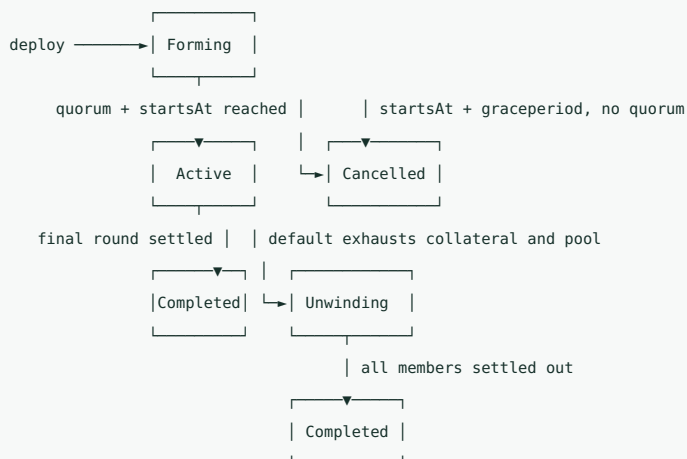
roundStart(r)      = startsAt + r × roundDuration
biddingOpensAt(r)  = roundStart(r)
biddingClosesAt(r) = roundStart(r) + biddingDuration
contributionDueAt(r) = roundStart(r) + roundDuration
settleableAt(r)    = contributionDueAt(r)

```

Constraint enforced at initialization: `biddingDuration < roundDuration`. Recommended default `biddingDuration = roundDuration / 4`.

A contribution submitted at or before `contributionDueAt(r)` is **on time**. There is no late window — after `contributionDueAt(r)` the round is settleable and collateral is slashed. A member may still contribute after the due date *if the round has not yet been settled*, and that counts as **late** (reputation penalty, no slash). Once settled, it is a **miss**.

2.8 Lifecycle



`Unwinding` is entered when a settlement cannot be made whole even after drawing the full available insurance coverage. In that state no further rounds run; members withdraw their remaining collateral and pending balances pro rata. It is a graceful failure mode, not a halt.

3. Contract inventory

Contract	Type	Deployed
<code>WhamCircleFactory</code>	Singleton	Once
<code>WhamCircle</code>	Implementation, cloned via EIP-1167	Once as implementation; one clone per circle
<code>WhamReputation</code>	Singleton, ERC-721 soulbound (ERC-5192)	Once
<code>WhamInsurancePool</code>	Singleton	Once
<code>MockUSDC</code>	Test token, 6 decimals, faucet	Testnet only

Dependencies: OpenZeppelin Contracts 5.x — `IERC20`, `SafeERC20`, `Clones`, `ECDSA`, `EIP712`, `ReentrancyGuard`, `Ownable2Step`, `ERC721`.

Toolchain: Foundry. `solc 0.8.28`, `optimizer_runs = 200`, `via_ir = true` (the settlement function is stack-heavy; without IR you will hit stack-too-deep and waste an afternoon restructuring it).

```
# foundry.toml

[profile.default]
solc = "0.8.28"
optimizer = true
optimizer_runs = 200
via_ir = true
evm_version = "cancun"
fs_permissions = [{ access = "read", path = "./deployments" }]

[profile.default.fuzz]
runs = 1024

[profile.ci.invariant]
runs = 256
depth = 64
```

Arbitrum notes that affect the design: - `block.timestamp` on Arbitrum tracks L1 time within a bounded drift. All deadlines here are hours or days apart, so timestamp precision is a non-issue. Do not use `block.number` for timing — L2 block numbers are not a reliable clock. - Calldata is the dominant gas cost. Keep function arguments tight; this is why bids are `uint16` basis points rather than token amounts. - Gas is cheap enough that explicit loops over `memberTarget ≤ 20` are the right call. Prefer auditable loops to clever accumulator math throughout.

4. Shared types, errors, and the factory

4.1 Enums

Ordinals are a cross-system contract. The backend casts these directly (Backend §4.3). Note the correction: Solidity needs `None = 0` on `MemberState` for default-value detection, which shifts every subsequent value by one relative to the backend enum. The backend must map explicitly rather than cast.

```
enum CircleState { Forming, Active, Completed, Unwinding, Cancelled }
//           0       1       2       3       4

enum MemberState { None, PendingCollateral, Active, PaidOut, Delinquent, Defaulted, Completed, Exited }
//           0       1       2       3       4       5       6       7

enum RoundState { Upcoming, Bidding, Funding, Settled, Covered }
//           0       1       2       3       4
```

`Covered` is `Settled` where `insuranceDrawn > 0`. It is a distinct state rather than a derived flag because the backend and UI treat it differently, and deriving it would mean reading two fields to answer one question.

4.2 Structs

```

struct CircleParams {
    address token;
    uint256 contributionPerRound;
    uint8  memberTarget;          // 3..20
    uint32 roundDuration;         // seconds
    uint32 biddingDuration;       // seconds, < roundDuration
    uint16 baseCollateralBps;      // informational; agent quote is authoritative
    uint16 insuranceSkimBps;       // <= MAX_SKIM_BPS
    uint16 maxDiscountBps;         // <= MAX_DISCOUNT_CEILING_BPS
    uint16 payoutCollateralBps;    // <= 10000; see §2.5
    bool   biddingEnabled;
    uint64 startsAt;
    address agent;                 // ERC-8004 registered agent authorised to sign quotes
}

struct Member {
    MemberState state;
    uint256 collateralRequired;    // raised on payout, see §2.5
    uint256 collateralPosted;
    uint256 collateralSlashed;
    uint8  joinIndex;             // deterministic tiebreak, and index into memberList
    uint8  roundsPaid;
    uint8  roundsMissed;
    uint8  roundsLate;
    int8   payoutRound;           // -1 until paid out
    uint16 winningDiscountBps;
    uint64 joinedAt;
}

struct Round {
    RoundState state;
    uint64 biddingClosesAt;
    uint64 contributionDueAt;
    uint64 settledAt;
    uint256 grossPot;
    address winner;
    uint16 winningDiscountBps;
    uint256 discountAmount;
    uint256 netPayout;
    uint256 redistributionPerMember;
    uint8  contributionsReceived;
    uint256 shortfall;
    uint256 collateralDrawn;
    uint256 insuranceDrawn;
}

/// EIP-712 payload signed by the agent, submitted by the member with join().
struct AgentQuote {
    address subject;
    address circle;
    uint256 collateralRequired;
    uint8  riskBand;              // 0=A .. 3=D
    uint256 nonce;
    uint64 expiresAt;
}

```

Struct packing. `Member` as written occupies 4 slots: three `uint256` (required, posted, slashed) plus one packed slot holding `state` (1) + `joinIndex` (1) + `roundsPaid` (1) + `roundsMissed` (1) + `roundsLate` (1) + `payoutRound` (1) + `winningDiscountBps` (2) + `joinedAt` (8) = 16 bytes. Declare the small fields contiguously in exactly that order or the compiler will not pack them.

4.3 Constants

```
uint16 internal constant BPS                = 10_000;
uint8  internal constant MIN_MEMBERS        = 3;
uint8  internal constant MAX_MEMBERS        = 20;
uint16 internal constant MIN_COLLATERAL_BPS = 7_500;    // 0.75x contribution
uint16 internal constant MAX_COLLATERAL_BPS = 30_000;    // 3.00x contribution
uint16 internal constant MAX_SKIM_BPS       = 300;       // 3.00%
uint16 internal constant MAX_DISCOUNT_CEILING_BPS = 2_500; // 25.00%
uint16 internal constant MIN_BID_INCREMENT_BPS = 25;      // 0.25%
uint32 internal constant MIN_ROUND_DURATION = 1 days;
uint32 internal constant MAX_ROUND_DURATION = 90 days;
uint32 internal constant QUORUM_GRACE_PERIOD = 7 days;
uint64 internal constant MAX_QUOTE_TTL      = 1 hours;
```

`MIN_COLLATERAL_BPS` and `MAX_COLLATERAL_BPS` are the hard bounds on what the agent can require. They are the reason a compromised agent key cannot drain or disable a circle (\$10.2). They match the clamp the backend applies in its scoring engine — but the contract enforces them independently, because a bound only the backend checks is not a bound.

4.4 Custom errors

Every error maps to user-facing copy in the frontend (Frontend §8.3). Errors carrying data must carry exactly what the message template needs.

```

// Lifecycle
error NotForming();
error NotActive();
error AlreadyStarted();
error CircleFull();
error QuorumNotReached();
error QuorumStillPossible();
error CircleNotComplete();

// Membership
error NotAMember();
error AlreadyMember();
error MemberNotActive();

// Quote / agent
error QuoteExpired();
error QuoteTtlTooLong();
error BadAgentSignature();
error QuoteSubjectMismatch();
error QuoteCircleMismatch();
error QuoteNonceMismatch(uint256 expected);
error CollateralOutOfBounds(uint256 min, uint256 max);

// Bidding
error BiddingDisabled();
error BiddingNotOpen();
error BiddingClosed();
error BidTooHigh(uint16 max);
error BidNotImproving(uint16 leading);
error AlreadyPaidOut();
error NoBidToWithdraw();

// Contribution
error NothingDue();
error AlreadyContributed();
error RoundNotOpen();

// Settlement
error RoundNotSettleable();
error RoundAlreadySettled();
error PreviousRoundUnsettled();

// Withdrawals
error NothingToWithdraw();
error CollateralLocked();
error PayoutNotOwed();

// Config / init
error AlreadyInitialized();
error InvalidParams();
error InvalidToken();
error FeeOnTransferToken();
error Unauthorized();

```

4.5 Events

The backend indexes exclusively from these (Backend §8.7). Indexed parameters are chosen so the indexer can filter by member address and by round without decoding data.

```

event CircleInitialized(address indexed token, uint8 memberTarget, uint256 contributionPerRound, uint64 startsAt);
event MemberJoined(address indexed member, uint256 collateral, uint8 riskBand, bytes32 decisionRef, uint8 joinIndex);
event MemberLeft(address indexed member, uint256 refunded);
event CircleStarted(uint64 at);
event CircleCancelled(uint64 at, string reason);

event BidPlaced(uint8 indexed round, address indexed bidder, uint16 discountBps);
event BidWithdrawn(uint8 indexed round, address indexed bidder);

event ContributionPaid(uint8 indexed round, address indexed member, uint256 amount, bool onTime);

event RoundSettled(
    uint8 indexed round,
    address indexed winner,
    uint16 discountBps,
    uint256 grossPot,
    uint256 netPayout,
    uint256 redistributedPerMember,
    uint256 insuranceDrawn
);
event CollateralDrawn(uint8 indexed round, address indexed member, uint256 amount, uint256 remaining);
event CollateralToppedUp(address indexed member, uint256 amount, uint256 newRequirement);
event ShortfallUncovered(uint8 indexed round, uint256 amount);

event PayoutClaimed(uint8 indexed round, address indexed member, uint256 amount);
event RewardsWithdrawn(address indexed member, uint256 amount);
event CollateralReleased(address indexed member, uint256 amount);

event MemberDefaulted(address indexed member, uint256 unpaidObligation);
event CircleCompleted(uint64 at);
event CircleUnwinding(uint64 at, uint256 uncoveredShortfall);

```

`decisionRef` in `MemberJoined` is `keccak256(abi.encode(quote))` — it ties the join to the agent’s ERC-8004 attestation (§9.3) so anyone can verify the deposit charged matches the decision published.

5. WhamCircleFactory

Deploys one minimal-proxy clone per circle and acts as the registry that the reputation contract and insurance pool trust.

5.1 Storage

```

contract WhamCircleFactory is Ownable2Step {
    address public immutable implementation;
    IWhamReputation public immutable reputation;
    IWhamInsurancePool public immutable insurancePool;

    mapping(address => bool) public isCircle;
    mapping(address => address[]) public circlesOfCreator;
    address[] public allCircles;

    mapping(address => bool) public allowedToken; // token allowlist
    address public defaultAgent; // used when params.agent == address(0)
    bool public paused;
}

```

The token allowlist is not optional. Without it anyone can deploy a circle around a malicious ERC-20 that reverts on transfer, returns false silently, or rebases — and every such circle appears in the browse list carrying the Wham name. Allowlist USDC on mainnet; USDC and `MockUSDC` on testnet.

5.2 createCircle

```
function createCircle(CircleParams calldata p, bytes32 salt) external returns (address circle);
```

Preconditions - `!paused` - `allowedToken[p.token]` - `p.memberTarget` in `[MIN_MEMBERS, MAX_MEMBERS]` - `p.contributionPerRound > 0` - `p.roundDuration` in `[MIN_ROUND_DURATION, MAX_ROUND_DURATION]` - `p.biddingDuration > 0 && p.biddingDuration < p.roundDuration` - `p.insuranceSkimBps <= MAX_SKIM_BPS` - `p.maxDiscountBps <= MAX_DISCOUNT_CEILING_BPS` - `p.payoutCollateralBps <= BPS` - `p.startsAt > block.timestamp` - `p.baseCollateralBps` in `[MIN_COLLATERAL_BPS, MAX_COLLATERAL_BPS]`

Effects 1. `circle = Clones.cloneDeterministic(implementation, keccak256(abi.encode(msg.sender, salt)))` 2. `WhamCircle(circle).initialize(p, address(this))` 3. `isCircle[circle] = true`; push to `circlesOfCreator[msg.sender]` and `allCircles` 4. `reputation.registerCircle(circle)` 5. `insurancePool.registerCircle(circle)`

Salting with `msg.sender` prevents one user from front-running another's chosen salt to occupy their predicted address. The backend's `predictAddress` must apply the same derivation.

Events: `CircleCreated(circle, creator, salt)` **Reverts:** `InvalidParams`, `InvalidToken`, `Unauthorized` (paused)

5.3 predictAddress

```
function predictAddress(address creator, bytes32 salt) external view returns (address);
```

Pure `Clones.predictDeterministicAddress`. The frontend shows this before the user signs (Frontend §8.1), and the backend verifies the deployed address matches (Backend §6.4).

5.4 Admin

```
function setAllowedToken(address token, bool allowed) external onlyOwner;
function setDefaultAgent(address agent) external onlyOwner;
function setPaused(bool p) external onlyOwner;
```

`Ownable2Step`, and the owner should be a multisig on mainnet. **The factory owner has no power over existing circles** — no pause, no parameter change, no fund access. Pausing only prevents new deployments. State this explicitly in the docs and on the agent transparency page; it is a stronger claim than most protocols can make and it costs nothing to be true.

6. `WhamCircle` — storage and initialization

6.1 Storage layout

```

contract WhamCircle is Initializable, ReentrancyGuard, EIP712Upgradeable {
    using SafeERC20 for IERC20;

    // — Config, written once in initialize —————
    IERC20 public token;
    uint256 public contributionPerRound;
    uint8 public memberTarget;
    uint32 public roundDuration;
    uint32 public biddingDuration;
    uint16 public baseCollateralBps;
    uint16 public insuranceSkimBps;
    uint16 public maxDiscountBps;
    uint16 public payoutCollateralBps;
    bool public biddingEnabled;
    uint64 public startsAt;
    address public agent;
    address public factory;
    IWhamReputation public reputation;
    IWhamInsurancePool public insurancePool;

    // — Lifecycle —————
    CircleState public state;
    uint8 public currentRound;
    uint8 public memberCount;
    uint64 public completedAt;

    // — Members —————
    address[] public memberList;
    mapping(address => Member) public memberOf;
    mapping(address => uint256) public agentNonce;

    // — Rounds —————
    mapping(uint8 => Round) internal _rounds;
    mapping(uint8 => mapping(address => bool)) public contributed;
    mapping(uint8 => mapping(address => uint16)) public bidOf;
    mapping(uint8 => address) public leadingBidder;
    mapping(uint8 => uint16) public leadingBidBps;

    // — Pull-payment balances —————
    mapping(address => uint256) public pendingPayout;
    mapping(address => uint256) public pendingRewards;

    // — Accounting totals, used by the solvency invariant ($11) —————
    uint256 public totalCollateralHeld;
    uint256 public totalRoundEscrow; // contributions received for the unsettled round
    uint256 public totalPendingPayout;
    uint256 public totalPendingRewards;
    uint256 public uncoveredShortfall;
}

```

The four `total*` accumulators exist solely so the solvency invariant is a cheap comparison rather than a loop. Keep them updated on every balance mutation; a drift between them and the sum of the mappings is the bug the invariant tests are designed to catch.

6.2 initialize

```
function initialize(CircleParams calldata p, address factory_) external initializer;
```

Preconditions: callable once, only by the factory (`msg.sender == factory_` and the factory validated params).

Effects 1. Copy every field of `p` into storage. 2. `__EIP712_init("WhamAgent", "1")` — the domain separator binds to

`address(this)`, which is why every circle needs its own quote and quotes are not portable between circles. 3. `state = Forming`, `currentRound = 0`. 4. **Fee-on-transfer check.** Before accepting any deposits the token must be proven to transfer exactly. Perform this at the first `join()` rather than here (there is nothing to transfer yet): measure `balanceOf(this)` before and after `safeTransferFrom` and revert `FeeOnTransferToken()` if the delta does not equal the requested amount. Do this check on **every** inbound transfer, not just the first — a token can turn fees on later.

Reverts: `AlreadyInitialized`, `InvalidParams`

6.3 View helpers

These exist because the frontend and backend read them constantly (Frontend §8.2). Implement them as real functions, not as things callers derive.

```
function config() external view returns (CircleParams memory);
function roundInfo(uint8 index) external view returns (Round memory);
function members() external view returns (address[] memory);

function roundStart(uint8 r) public view returns (uint64);
function biddingClosesAt(uint8 r) public view returns (uint64);
function contributionDueAt(uint8 r) public view returns (uint64);

function grossPotFull() public view returns (uint256); // memberTarget * contributionPerRound

/// Net payout a bid of `discountBps` would produce this round, before any collateral top-up.
function quotePayout(uint8 round, uint16 discountBps) external view returns (uint256);

/// The top-up `who` would owe if they won `round` — powers the bid composer's live panel.
function quoteTopUp(uint8 round, address who) external view returns (uint256);

function amountDue(uint8 round, address who) external view returns (uint256);
function withdrawableCollateral(address who) external view returns (uint256);
function remainingObligation(address who) public view returns (uint256);
function eligibleBidders() external view returns (uint256 count);
function isSettleable(uint8 round) external view returns (bool);
```

`quotePayout` and `quoteTopUp` must use **exactly** the same arithmetic as `settleRound`, extracted into shared internal pure functions. If the preview and the settlement can disagree by even one base unit, the bid composer is lying to users about what they will receive. Factor the math out once and call it from both; a test should assert equality across a fuzzed input space.

7. WhamCircle — functions

Each specified as **signature** → **preconditions** → **effects** → **events** → **reverts**. Implement in this order; later functions depend on earlier state being correct.

7.1 join

```
function join(AgentQuote calldata q, bytes calldata agentSig) external nonReentrant;
```

The member submits a quote the agent signed off-chain (§9.2). The agent sends no transaction and pays no gas.

Preconditions - `state == Forming` → `else NotForming()` - `memberOf[msg.sender].state == None` → `else AlreadyMember()` - `memberCount < memberTarget` → `else CircleFull()` - `q.subject == msg.sender` → `else QuoteSubjectMismatch()` - `q.circle == address(this)` → `else QuoteCircleMismatch()` - `q.nonce == agentNonce[msg.sender]` → `else QuoteNonceMismatch(expected)` - `q.expiresAt > block.timestamp` → `else QuoteExpired()` - `q.expiresAt <= block.timestamp + MAX_QUOTE_TTL` → `else QuoteTtlTooLong()` - `q.collateralRequired` within `[contributionPerRound × MIN_COLLATERAL_BPS / BPS, contributionPerRound × MAX_COLLATERAL_BPS / BPS]` → `else CollateralOutOfBounds(min, max)` - Signature recovers to `agent` → `else BadAgentSignature()`

Effects 1. `agentNonce[msg.sender]++` — **before** any external call. This is the replay guard; incrementing it late is a reentrancy hole. 2. Pull `q.collateralRequired` via `_pullExact()` (§7.11). 3. Write the `Member` record: `state = Active`, `collateralRequired = collateralPosted = q.collateralRequired`, `joinIndex = memberCount`, `payoutRound = -1`, `joinedAt = block.timestamp`. 4. `memberList.push(msg.sender)`; `memberCount++`; `totalCollateralHeld += q.collateralRequired`. 5. If `memberCount == memberTarget` &&

`block.timestamp >= startsAt`, call `_start()` internally.

Note the deliberate difference from the companion documents. Frontend §9.9 and Backend §6.5 both describe `join()` pulling collateral **and the first contribution** together. Do not do that. Collateral is posted at `join`; the first contribution is paid via `contribute(0)` after the circle starts. Reasons: a circle that never reaches quorum must refund cleanly with no round accounting to unwind, and the first round's bidding window opens *after* the start, so a contribution collected at `join` would sit in escrow for an indeterminate period. The frontend's approval step should approve `collateral + contributionPerRound` (so the user signs one approval for both actions) but `join()` transfers only the collateral.

Events: `MemberJoined(member, collateral, riskBand, keccak256(abi.encode(q)), joinIndex)`, plus `CircleStarted` if it started.

7.2 `leaveBeforeStart`

```
function leaveBeforeStart() external nonReentrant;
```

Preconditions: `state == Forming`; caller is a member.

Effects 1. Credit `collateralPosted` to `pendingRewards[msg.sender]` (pull payment — do not transfer here). 2. `totalCollateralHeld -= collateralPosted`. 3. Set `state = Exited`, zero the collateral fields. 4. Remove from `memberList` by swapping in the last element and **reassigning that member's `joinIndex`**. Forgetting the reassignment silently corrupts the no-bid tiebreak. 5. `memberCount--`.

Events: `MemberLeft(member, refunded)` **Reverts:** `NotForming`, `NotAMember`

7.3 `start`

```
function start() external;
```

Permissionless. Also called internally by the last `join`.

Preconditions: `state == Forming`, `memberCount == memberTarget`, `block.timestamp >= startsAt` → else `QuorumNotReached()`.

Effects: `state = Active`; initialize round 0 (`state = Bidding`, timestamps from §2.7). If `!biddingEnabled`, round 0 opens directly in `Funding`.

Events: `CircleStarted(block.timestamp)`

7.4 `cancel`

```
function cancel() external;
```

Permissionless escape hatch for a circle that never fills.

Preconditions: `state == Forming`, `block.timestamp > startsAt + QUORUM_GRACE_PERIOD` → else `QuorumStillPossible()`.

Effects: `state = Cancelled`; for each member credit `collateralPosted` to `pendingRewards`; zero `totalCollateralHeld`.

Events: `CircleCancelled(block.timestamp, "quorum not reached")`

7.5 `placeBid`

```
function placeBid(uint8 round, uint16 discountBps) external;
```

Preconditions - `state == Active`, `round == currentRound` - `biddingEnabled` → else `BiddingDisabled()` - `_rounds[round].state == Bidding` and `block.timestamp <= biddingClosesAt(round)` → else `BiddingClosed()` - Caller is a member with `state ∈ {Active, Delinquent}` → else `MemberNotActive()` - `memberOf[msg.sender].payoutRound < 0` → else `AlreadyPaidOut()` - `discountBps <= maxDiscountBps` → else `BidTooHigh(maxDiscountBps)` - `discountBps >= leadingBidBps[round] + MIN_BID_INCREMENT_BPS` → else `BidNotImproving(leadingBidBps[round])`

Effects: `bidOf[round][msg.sender] = discountBps`; `leadingBidder[round] = msg.sender`; `leadingBidBps[round] = discountBps`.

Events: `BidPlaced(round, bidder, discountBps)`

Design note. Strict improvement by `MIN_BID_INCREMENT_BPS` makes ties impossible on-chain, which resolves open question #3 from both companion documents: there is no tiebreak rule because there are no ties. It also prevents a gas-war of one-basis-point outbids. Tell the backend it can drop its tie-handling logic.

A member may raise their own bid; there is no reason to forbid it and forbidding it creates a confusing dead end when someone mistypes.

7.6 withdrawBid

```
function withdrawBid(uint8 round) external;
```

Preconditions: bidding open for `round`; `bidOf[round][msg.sender] > 0` $\rightarrow$ else `NoBidToWithdraw()`.

Effects: zero the caller's bid. If they were the leader, **recompute the leader by looping** `memberList` for the highest remaining bid among eligible members. With `memberTarget  $\leq$  20` this is cheap and correct; maintaining a sorted structure to avoid it is not worth the surface area.

Events: `BidWithdrawn(round, bidder)`

7.7 contribute

```
function contribute(uint8 round) external nonReentrant;
```

Preconditions - `state == Active`, `round == currentRound` - `_rounds[round].state  $\in$  {Bidding, Funding}` $\rightarrow$ else `RoundNotOpen()` - Caller is a member with `state  $\in$  {Active, PaidOut, Delinquent}` $\rightarrow$ else `NotAMember()` - `!contributed[round][msg.sender]` $\rightarrow$ else `AlreadyContributed()`

Effects 1. `bool onTime = block.timestamp <= contributionDueAt(round)` 2. **Consume pending rewards first.** `uint256 fromRewards = min(pendingRewards[msg.sender], contributionPerRound)`; deduct it from `pendingRewards` and `totalPendingRewards`. Pull only the remainder from the wallet via `_pullExact()`. 3. `contributed[round][msg.sender] = true`; `_rounds[round].contributionsReceived++`; `totalRoundEscrow += contributionPerRound`. 4. `member.roundsPaid++`; if `!onTime` then `member.roundsLate++`. 5. `reputation.recordContribution(msg.sender, onTime)`. 6. If the member was `Delinquent` and now owes nothing outstanding, restore `state` to `Active` (or `PaidOut` if `payoutRound >= 0`).

Events: `ContributionPaid(round, member, contributionPerRound, onTime)`

The rewards-netting behaviour in step 2 is a real UX win — a member with 8.50 USDC of accumulated redistribution pays 41.50 from their wallet instead of 50.00, in one transaction. Surface it in the frontend's payment sheet: *"8.50 USDC of your redistribution credit will be used. 41.50 USDC from your wallet."*

Members who have already been paid out **must still contribute every remaining round**. This is the obligation the payout-collateral top-up (\$2.5) partially secures, and the single most important thing for the frontend to keep saying.

7.8 settleRound

```
function settleRound(uint8 round) external nonReentrant;
```

Permissionless. The backend keeper calls it (Backend §11.1); any member can too.

Preconditions - `state == Active`, `round == currentRound` - `_rounds[round].state != Settled && != Covered` $\rightarrow$ else `RoundAlreadySettled()` - `block.timestamp >= contributionDueAt(round)` $\rightarrow$ else `RoundNotSettleable()`

Effects — implement in exactly this order (§2.6):

```
// 1. Winner
address winner = leadingBidder[round];
uint16 disc = leadingBidBps[round];
if (winner == address(0)) {
    (winner, disc) = (_lowestIndexUnpaidMember(), 0); // deterministic fallback
}

// 2. Collect from defaulters
uint256 pot = totalRoundEscrow; // what actually arrived via contribute()
uint256 shortfall;
uint256 drawnTotal;
for (uint8 i; i < memberList.length; ++i) {
    address m = memberList[i];
    if (contributed[round][m]) continue;

    Member storage mem = memberOf[m];
    uint256 available = mem.collateralPosted - mem.collateralSlashed;
    uint256 drawn = available >= contributionPerRound ? contributionPerRound : available;
```

```

    if (drawn > 0) {
        mem.collateralSlashed += drawn;
        totalCollateralHeld -= drawn;
        pot += drawn;
        drawnTotal += drawn;
        emit CollateralDrawn(round, m, drawn, mem.collateralPosted - mem.collateralSlashed);
    }
    if (drawn < contributionPerRound) shortfall += contributionPerRound - drawn;

    mem.roundsMissed++;
    mem.state = (mem.collateralPosted - mem.collateralSlashed == 0)
        ? MemberState.Defaulted
        : MemberState.Delinquent;
    if (mem.state == MemberState.Defaulted) {
        emit MemberDefaulted(m, remainingObligation(m));
        reputation.recordDefault(m);
    } else {
        reputation.recordMiss(m);
    }
}

// 3. Insurance draw
uint256 insuranceDrawn;
if (shortfall > 0) {
    insuranceDrawn = insurancePool.claim(shortfall); // returns what it could actually pay
    pot += insuranceDrawn;
    shortfall -= insuranceDrawn;
}

// 4. Skim, discount, net
uint256 skim = (pot * insuranceSkimBps) / BPS;
uint256 discAmt = (pot * disc) / BPS;
uint256 net = pot - skim - discAmt;

// 5. Redistribution to members still waiting
uint256 waiting = uint256(memberTarget) - uint256(round) - 1;
uint256 perMember = waiting > 0 ? discAmt / waiting : 0;
uint256 dust = discAmt - (perMember * waiting);

// 6. Winner's collateral top-up ($2.5) – funded from the payout
uint256 topUp = _requiredCollateralAtPayout(winner, round);
if (topUp > 0) {
    net -= topUp; // reverts on underflow: see the note below
    memberOf[winner].collateralPosted += topUp;
    memberOf[winner].collateralRequired += topUp;
    totalCollateralHeld += topUp;
    emit CollateralToppedUp(winner, topUp, memberOf[winner].collateralRequired);
}

```

The `net -= topUp` underflow. In a badly under-funded round the top-up can exceed the net payout. Do not let it revert — that would make the round unseizable and freeze the circle. Cap it: `topUp = min(topUp, net)` and accept partial collateralisation. Record the gap so it is visible.

```

// 7-9. Credit balances, pay the pool
pendingPayout[winner] += net;    totalPendingPayout += net;
for (each waiting member) { pendingRewards[m] += perMember; }
totalPendingRewards += perMember * waiting;
token.safeTransfer(address(insurancePool), skim + dust);
insurancePool.notifySkim(round, skim + dust);

// 10. State updates
memberOf[winner].payoutRound      = int8(round);
memberOf[winner].winningDiscountBps = disc;
memberOf[winner].state            = MemberState.PaidOut;
_rounds[round] = Round({ ...all computed values..., settledAt: block.timestamp,
                        state: insuranceDrawn > 0 ? RoundState.Covered : RoundState.Settled });
totalRoundEscrow = 0;

if (shortfall > 0) {
    uncoveredShortfall += shortfall;
    emit ShortfallUncovered(round, shortfall);
    state = CircleState.Unwinding;
    emit CircleUnwinding(block.timestamp, uncoveredShortfall);
    return; // no further rounds
}

// 11. Advance
if (round + 1 == memberTarget) {
    state = CircleState.Completed;
    completedAt = uint64(block.timestamp);
    _finalizeReputation();
    emit CircleCompleted(completedAt);
} else {
    currentRound = round + 1;
    _rounds[currentRound].state = biddingEnabled ? RoundState.Bidding : RoundState.Funding;
    _rounds[currentRound].biddingClosesAt = biddingClosesAt(currentRound);
    _rounds[currentRound].contributionDueAt = contributionDueAt(currentRound);
}

```

Events: `RoundSettled(...)` plus every `CollateralDrawn`, `CollateralToppedUp`, `MemberDefaulted`, `ShortfallUncovered` emitted along the way.

Idempotency. `settleRound` must be safe to call twice in the same block from two keepers. The `RoundAlreadySettled()` check plus `nonReentrant` handles it, but write the test — the backend keeper retries, and a member may click the fallback button at the same moment.

`_requiredCollateralAtPayout:`

```

function _requiredCollateralAtPayout(address who, uint8 round) internal view returns (uint256 topUp) {
    uint256 remaining = uint256(memberTarget - round - 1) * contributionPerRound;
    uint256 required = (remaining * payoutCollateralBps) / BPS;
    Member storage m = memberOf[who];
    uint256 held = m.collateralPosted - m.collateralSlashed;
    topUp = required > held ? required - held : 0;
}

```

7.9 claimPayout

```
function claimPayout(uint8 round) external nonReentrant;
```

Preconditions: `memberOf[msg.sender].payoutRound == int8(round) → else PayoutNotOwed(); pendingPayout[msg.sender] > 0 → else NothingToWithdraw().`

Effects: zero the balance (checks-effects-interactions), decrement `totalPendingPayout`, `safeTransfer`.

Events: `PayoutClaimed(round, member, amount)`

The `round` argument is redundant to the contract but valuable to the indexer and the frontend's success screen. Keep it and validate it rather than ignoring it.

7.10 `withdrawRewards`

```
function withdrawRewards() external nonReentrant;
```

Drains `pendingRewards` — discount redistribution, plus refunds from `leaveBeforeStart` and `cancel`. Available at any time; this money is not locked.

Events: `RewardsWithdrawn(member, amount)`

This function is missing from Frontend \$8.3. The member circle view needs a corresponding card in its action stack: “You’ve earned 24.50 USDC from other members’ discounts.” with a withdraw action, and the dashboard’s `PositionStrip` should surface the balance.

7.11 `withdrawCollateral`

```
function withdrawCollateral() external nonReentrant;
```

Preconditions - `state ∈ {Completed, Cancelled, Unwinding} → else CollateralLocked()` - Caller is a member; `collateralPosted > collateralSlashed → else NothingToWithdraw()` - If `Completed`: caller has contributed to all `memberTarget` rounds. A member who missed rounds has already had those amounts slashed, so the remainder is theirs — but assert the accounting explicitly rather than assuming.

Effects: compute `amount = collateralPosted - collateralSlashed`; zero both fields; `totalCollateralHeld -= amount`; set `member.state = Completed`; `safeTransfer`.

Unwinding is pro rata. When `uncoveredShortfall > 0` the contract holds less than members are owed. Reduce each withdrawal proportionally:

```
uint256 owed  = m.collateralPosted - m.collateralSlashed;
uint256 total = totalCollateralHeld;
uint256 avail = token.balanceOf(address(this)) - totalPendingPayout - totalPendingRewards;
uint256 amount = total == 0 ? 0 : (owed * avail) / total;
```

Events: `CollateralReleased(member, amount)`

7.12 `releaseExcessCollateral` (optional — ship if time permits)

```
function releaseExcessCollateral() external nonReentrant;
```

As rounds are paid, a member's remaining obligation shrinks and their required collateral with it. This lets them withdraw the excess mid-circle instead of at completion.

```
uint256 required = _requiredCollateralNow(msg.sender); // max(joinCollateral scaled, payout-contingent)
uint256 held     = m.collateralPosted - m.collateralSlashed;
uint256 excess   = held > required ? held - required : 0;
```

Do not ship this unless the core path is finished and tested. It touches the collateral accounting that every other function depends on, and a bug here drains the circle. It is a capital-efficiency nicety, not a mechanism.

7.13 Internal helpers

```
/// Pull exactly `amount`, rejecting fee-on-transfer and rebasing tokens.
function _pullExact(address from, uint256 amount) internal {
    uint256 before = token.balanceOf(address(this));
    token.safeTransferFrom(from, address(this), amount);
    if (token.balanceOf(address(this)) - before != amount) revert FeeOnTransferToken();
}

function _lowestIndexUnpaidMember() internal view returns (address);
function _finalizeReputation() internal; // one call per member into WhamReputation
```

`_finalizeReputation` loops `memberList` and calls `reputation.recordCompletion(member, roundsPaid, roundsLate, roundsMissed)`. With 20 members this is 20 external calls in one transaction — measure the gas. If it exceeds comfort, split it into a permissionless `finalizeReputation(uint8 from, uint8 to)` that anyone can call in batches after completion, and have the backend keeper drive it. Decide this from a real gas measurement, not from a guess.

8. WhamReputation

A soulbound ERC-721 that holds each member's savings record. **The score is computed on-chain**, from constants published in the contract, using data the circles report as it happens. This is correction #1 from §1: a reputation the backend can rewrite is not a reputation, and everything the score depends on already lives on-chain.

8.1 Storage and constants

```
contract WhamReputation is ERC721, Ownable2Step, IERC5192 {
    struct Record {
        uint16 score;           // 0..1000
        uint16 circlesCompleted;
        uint16 circlesActive;
        uint16 roundsOnTime;
        uint16 roundsLate;
        uint16 roundsMissed;
        uint16 defaults;
        uint64 firstSeenAt;
    }

    mapping(address => Record) public recordOf;
    mapping(address => uint256) public tokenIdOf;
    mapping(address => bool) public isRegisteredCircle;

    IWhamCircleFactory public immutable factory;
    uint256 private _nextTokenId = 1;

    // Published score movements – these are product copy (Frontend §9.16)
    int16 public constant DELTA_ON_TIME      = 8;
    int16 public constant DELTA_LATE        = -15;
    int16 public constant DELTA_MISSED      = -60;
    int16 public constant DELTA_COMPLETE_CLEAN = 120;
    int16 public constant DELTA_COMPLETE_MESSY = 40;
    int16 public constant DELTA_DEFAULT    = -400;
    uint16 public constant MAX_SCORE      = 1000;
}
```

8.2 Access control

```
modifier onlyCircle() {
    if (!factory.isCircle(msg.sender)) revert Unauthorized();
    _;
}
```

Authorisation flows from the factory registry, not from an owner-managed allowlist. That means the owner cannot grant reputation-writing power to an arbitrary address — only a circle deployed by the factory can move a score. Worth stating publicly; it is the difference between an on-chain reputation and a database with extra steps.

8.3 Write functions

```

function recordContribution(address member, bool onTime) external onlyCircle;
function recordMiss(address member) external onlyCircle;
function recordDefault(address member) external onlyCircle;
function recordCompletion(address member, uint16 roundsPaid, uint16 roundsLate, uint16 roundsMissed) external onlyCircle;
function registerCircle(address circle) external; // called by factory at deploy
function noteJoin(address member) external onlyCircle; // mints on first ever join

```

`recordCompletion` applies `DELTA_COMPLETE_CLEAN` when `roundsLate == 0 && roundsMissed == 0`, else `DELTA_COMPLETE_MESSY`, and increments `circlesCompleted`.

Score application is saturating, never wrapping:

```

function _apply(address member, int16 delta) internal {
    Record storage r = recordOf[member];
    int256 next = int256(uint256(r.score)) + int256(delta);
    if (next < 0) next = 0;
    if (next > int256(uint256(MAX_SCORE))) next = int256(uint256(MAX_SCORE));
    uint16 old = r.score;
    r.score = uint16(uint256(next));
    emit ScoreUpdated(member, old, r.score, delta);
}

```

8.4 Soulbound enforcement

Implement ERC-5192 (`locked(tokenId)` returns `true` always, `Locked` event on mint) and block every transfer path:

```

function _update(address to, uint256 tokenId, address auth) internal override returns (address) {
    address from = _ownerOf(tokenId);
    if (from != address(0) && to != address(0)) revert Soulbound(); // mint and burn only
    return super._update(to, tokenId, auth);
}

function approve(address, uint256) public pure override { revert Soulbound(); }
function setApprovalForAll(address, bool) public pure override { revert Soulbound(); }

```

Overriding `_update` in OpenZeppelin 5.x covers `transferFrom`, `safeTransferFrom`, and every internal path. Blocking `approve` and `setApprovalForAll` as well is belt-and-braces, but it also means marketplaces surface the restriction clearly rather than listing a token that can never settle.

8.5 Views and metadata

```

function scoreOf(address) external view returns (uint16);
function bandOf(address) external view returns (uint8); // 0=A..3=D, thresholds below
function tierOf(address) external view returns (uint8); // 0=Starter, 1=Standard, 2=Trusted
function statsOf(address) external view returns (uint16 completed, uint16 onTime, uint16 late, uint16 missed, uint16 defaults);
function tokenURI(uint256) public view override returns (string memory);

```

Tier thresholds, matching Backend §9.2:

Tier	Requires
Starter	default
Standard	<code>circlesCompleted >= 1 && score >= 250</code>
Trusted	<code>circlesCompleted >= 3 && score >= 600 && defaults == 0</code>

`tokenURI` should return a fully on-chain base64 data URI containing an SVG rendering the score, band, and completed-circle count. Do not point at an API. This token is the thing members share as proof of their record (Component brief §5.4, `AttestationCard`) and it must still render in five years if the backend is gone. It is also, in practice, the artifact judges will click on.

9. The agent quote and ERC-8004

9.1 Why a signed quote rather than an agent transaction

The original design had the agent call `setCollateralRequirement(member, amount)` before each join. Signed quotes are strictly better:

	Agent transaction	Signed quote
Gas	Agent pays, per member	Nobody pays extra
Liveness	Join blocked if the agent is down	Quote works for its TTL regardless
Race	Two joins race the agent's nonce	Each quote carries its own nonce
Trust	Agent has a live write path into the circle	Agent has no write path at all
Auditability	A transaction to correlate	The signed payload is the record

The agent never transacts against a circle. Its only on-chain writes are ERC-8004 attestations to a registry the circles do not read.

9.2 EIP-712 domain and typehash

```
bytes32 internal constant AGENT_QUOTE_TYPEHASH = keccak256(
    "AgentQuote(address subject,address circle,uint256 collateralRequired,uint8 riskBand,uint256 nonce,uint64 expiresAt)"
);

function _verifyQuote(AgentQuote calldata q, bytes calldata sig) internal view {
    bytes32 structHash = keccak256(abi.encode(
        AGENT_QUOTE_TYPEHASH, q.subject, q.circle, q.collateralRequired, q.riskBand, q.nonce, q.expiresAt
    ));
    address signer = ECDSA.recover(_hashTypedDataV4(structHash), sig);
    if (signer != agent) revert BadAgentSignature();
}
```

Domain: `name: "WhamAgent", version: "1", chainId, verifyingContract: address(this)`.

This must match the backend's `AgentQuoteMessage` exactly — field order, field types, domain string. The backend spec (§7.5) declares `expiresAt` as `uint64` and `collateralRequired` as `uint256`; a mismatch produces a signature that recovers to a wrong address and reverts with `BadAgentSignature()` only after the user has paid gas.

Write this test before anything else that depends on it:

```
function test_QuoteSignedByBackendFormat_IsAcceptedOnChain() public {
    AgentQuote memory q = AgentQuote({ subject: alice, circle: address(circle),
        collateralRequired: 100e6, riskBand: 1, nonce: 0, expiresAt: uint64(block.timestamp + 900) });
    bytes32 digest = circle.hashQuote(q); // expose a view helper for exactly this
    (uint8 v, bytes32 r, bytes32 s) = vm.sign(agentPk, digest);
    vm.prank(alice);
    circle.join(q, abi.encodePacked(r, s, v));
    assertEq(circle.memberCount(), 1);
}
```

Expose `hashQuote(AgentQuote)` as a public view so the backend can assert its own digest matches the contract's in an integration test. Twenty lines of contract surface that eliminates the most likely demo-day failure.

9.3 ERC-8004 integration

The circles do **not** read the ERC-8004 registries at runtime. Adding a registry lookup to `join()` would make every join depend on a third-party contract for no security benefit — the agent address is already fixed at deployment and the collateral bounds are enforced regardless.

The integration is at two points, neither on a hot path:

At deployment. The factory optionally asserts the configured agent resolves to a registered ERC-8004 identity:

```
(uint256 agentId, ) = identityRegistry.resolveByAddress(p.agent);
if (agentId == 0) revert AgentNotRegistered();
```

Make this a factory-level toggle so a local test chain without the registries deployed still works.

After each decision. The agent posts an attestation to the ERC-8004 `ReputationRegistry` referencing the decision hash (Backend §7.6). The reference is:

```
decisionRef = keccak256(abi.encode(quote))
```

which is exactly the value the circle emits in `MemberJoined`. That is the link that makes the whole claim checkable: anyone can take the `decisionRef` from a join event, find the matching attestation on the registry, fetch the published decision JSON from the API, hash it, and confirm all three agree. Document this chain explicitly in the README — it is the ERC-8004 story, and it is worth nothing if nobody can follow it.

10. `WhamInsurancePool`

10.1 Storage

```
contract WhamInsurancePool is Ownable2Step, ReentrancyGuard {
    IERC20 public immutable token;
    IWhamCircleFactory public immutable factory;

    uint256 public totalReserves;
    uint256 public committedCoverage;
    uint256 public claimsPaidCount;
    uint256 public claimsPaidTotal;

    mapping(address => uint256) public skimReceivedFrom;
    mapping(address => uint256) public claimsPaidTo;
    mapping(address => bool) public isRegistered;

    uint16 public perCircleCapBps = 2_000; // one circle may draw at most 20% of reserves
}
```

10.2 `claim`

```
function claim(uint256 amount) external onlyCircle nonReentrant returns (uint256 paid);
```

Effects

```
uint256 available = totalReserves;
uint256 cap       = (totalReserves * perCircleCapBps) / BPS;
uint256 alreadyPaid = claimsPaidTo[msg.sender];
uint256 headroom   = cap > alreadyPaid ? cap - alreadyPaid : 0;

paid = amount;
if (paid > headroom) paid = headroom;
if (paid > available) paid = available;

if (paid > 0) {
    totalReserves -= paid;
    claimsPaidTo[msg.sender] += paid;
    claimsPaidTotal += paid;
    claimsPaidCount++;
    token.safeTransfer(msg.sender, paid);
    emit ClaimPaid(msg.sender, paid, amount - paid);
}
```

Returning a partial payment rather than reverting is deliberate. A pool that reverts when it cannot cover a claim in full would make `settleRound` unseizable and freeze every member's funds in a circle that already has a defaulter. Partial coverage plus a graceful `Unwinding` transition is strictly better than a frozen circle, and the shortfall is emitted so it is visible.

The per-circle cap stops one large circle with several defaults from draining a pool that a hundred small circles depend on. 20% is a starting point; tune it against the demo data.

10.3 Other functions

```
function notifySkim(uint8 round, uint256 amount) external onlyCircle; // accounting only; tokens already sent
function registerCircle(address circle) external; // factory only
function availableCoverage() external view returns (uint256);
function coverageFor(address circle) external view returns (uint256);
function fund(uint256 amount) external; // anyone may seed the pool
function setPerCircleCapBps(uint16) external onlyOwner;
```

`fund()` matters for the demo: the pool starts empty, and a pool with no reserves cannot demonstrate the third layer of the default waterfall. Seed it during deployment (§14.3).

There is no withdrawal function and there must never be one. The owner cannot remove reserves. This is the claim the insurance page makes, and it should be true at the bytecode level rather than as a policy. Say so in a comment at the top of the contract, because that is where a reviewer will look for it.

11. Security model

11.1 Threat table

Threat	Mitigation
Reentrancy on any token movement	All outbound transfers are pull-based; <code>nonReentrant</code> on every state-changing external; strict checks-effects-interactions. <code>join</code> increments the nonce before <code>transferFrom</code> .
Agent key compromise	Bounded by <code>MIN_COLLATERAL_BPS / MAX_COLLATERAL_BPS</code> in <code>join</code> . A compromised agent can set collateral anywhere in <code>[0.75×, 3×]</code> — it cannot zero it, cannot make it unpayable, and cannot touch funds. Worst case is mispriced risk on new joins only.
Quote replay	Per-subject nonce read from contract storage; <code>circle</code> and <code>subject</code> bound into the signed struct; TTL capped at <code>MAX_QUOTE_TTL</code> .
Signature malleability	OpenZeppelin <code>ECDSA.recover</code> rejects high- <code>s</code> and zero addresses.
Fee-on-transfer / rebasing token	<code>pullExact</code> balance-delta check on every inbound transfer; factory token allowlist.
Griefing via bid spam	<code>MIN_BID_INCREMENT_BPS</code> forces meaningful improvements; bids cost only gas and cannot block settlement.
Settlement DoS	<code>settleRound</code> is permissionless, has no unbounded loop (<code>memberTarget ≤ 20</code>), and never reverts on a defaulter. Insurance shortfall degrades to <code>Unwinding</code> rather than reverting.
Stuck circle from a failing external call	Reputation calls happen inside settlement. If <code>WhamReputation</code> ever reverted, settlement would freeze. Wrap reputation calls in <code>try/catch</code> and emit <code>ReputationCallFailed</code> — the savings mechanism must never be hostage to the scoring one.
Winner drains and abandons	Payout-contingent collateral top-up (\$2.5) + discount bidding pricing the risk + permanent on-chain default record.
Front-running a join to take the last seat	Not meaningfully harmful — a seat is an obligation as much as an opportunity. No mitigation needed; do not add commit-reveal for this.
Front-running the final bid	Bids are public and the auction closes on a timestamp. Sniping in the last block is possible and acceptable — it is an ascending auction, not a sealed one. Document the behaviour rather than engineering around it.

Factory owner overreach	Owner can pause new deployments and manage the token allowlist. Owner has no function that touches an existing circle, its funds, or its members.
Insurance pool drain	Per-circle cap; <code>onlyCircle</code> gated on the factory registry; no withdrawal function exists.

11.2 Invariants

Write these as Foundry invariant tests. They are worth more than any individual unit test.

```

/// INV-1 Solvency: the contract always holds at least what it owes.
token.balanceOf(circle) >= totalCollateralHeld + totalRoundEscrow
    + totalPendingPayout + totalPendingRewards;

/// INV-2 Collateral accounting: the accumulator matches the members.
totalCollateralHeld ==  $\Sigma$  (m.collateralPosted - m.collateralSlashed) over all members;

/// INV-3 Slash bound: nobody is ever slashed beyond what they posted.
 $\forall$  m: m.collateralSlashed <= m.collateralPosted;

/// INV-4 One payout each: no member is paid twice, no round pays twice.
 $\forall$  m: m.payoutRound  $\in$  [-1, memberTarget-1];
 $\forall$  r settled: exactly one member has payoutRound == r;

/// INV-5 Round monotonicity: rounds settle strictly in order.
 $\forall$  r < currentRound: _rounds[r].state  $\in$  {Settled, Covered};

/// INV-6 Conservation: every unit in is a unit out or a unit held.
 $\Sigma$  contributions +  $\Sigma$  collateral in +  $\Sigma$  insurance drawn
    ==  $\Sigma$  payouts +  $\Sigma$  rewards +  $\Sigma$  skim to pool +  $\Sigma$  collateral returned + balanceOf(circle);

/// INV-7 Bid validity: the recorded leader really is the highest eligible bid.
leadingBidBps[r] == max(bidOf[r][m]) over members with payoutRound < 0;

/// INV-8 Bounded collateral at join: the agent cannot escape the bounds.
 $\forall$  m: m.collateralPosted at join  $\in$  [C*MIN_COLLATERAL_BPS/BPS, C*MAX_COLLATERAL_BPS/BPS];

```

INV-1 and INV-6 are the ones that matter. If either can be broken by any sequence of calls, funds can be lost. Run them with `runs = 256, depth = 64` in CI and higher locally.

11.3 Deliberate non-goals

State these in the README so a reviewer knows they were decisions, not oversights:

- **No upgradeability on circles.** A deployed circle's rules cannot change. Members joined on those terms. The factory can point at a new implementation for *future* circles only.
- **No admin pause on an active circle.** Nobody, including the deployer, can freeze a running circle's funds.
- **No governance token, no protocol fee, no treasury.** The only value flow out of a circle is the insurance skim, which goes to a pool that pays members.
- **No oracle, no randomness, no external price feed.** The system has no dependency that can be manipulated or go stale.

12. Test matrix

Foundry. Every row is a test that must exist and pass before the demo build is cut.

12.1 Unit — `WhamCircle` lifecycle

#	Test
1	<code>join</code> with a valid quote posts collateral and records the member

2	<code>join</code> reverts <code>QuoteExpired</code> past <code>expiresAt</code>
3	<code>join</code> reverts <code>QuoteTtlTooLong</code> when TTL exceeds one hour
4	<code>join</code> reverts <code>BadAgentSignature</code> when signed by a non-agent key
5	<code>join</code> reverts <code>QuoteSubjectMismatch</code> when Alice submits Bob's quote
6	<code>join</code> reverts <code>QuoteCircleMismatch</code> when a quote from circle A is used on circle B
7	<code>join</code> reverts <code>QuoteNonceMismatch</code> on replay of a consumed quote
8	<code>join</code> reverts <code>CollateralOutOfBounds</code> at 0.5× and at 4× contribution
9	<code>join</code> reverts <code>CircleFull</code> on the <code>memberTarget + 1</code> th member
10	<code>leaveBeforeStart</code> refunds in full and reassigns the swapped member's <code>joinIndex</code>
11	<code>start</code> reverts <code>QuorumNotReached</code> below quorum, succeeds at quorum
12	<code>cancel</code> reverts <code>QuorumStillPossible</code> before grace, succeeds after, refunds everyone

12.2 Unit — bidding

#	Test
13	<code>placeBid</code> sets the leader; a higher bid replaces it
14	<code>placeBid</code> reverts <code>BidNotImproving</code> at equal and at sub-increment improvement
15	<code>placeBid</code> reverts <code>BidTooHigh</code> above <code>maxDiscountBps</code>
16	<code>placeBid</code> reverts <code>AlreadyPaidOut</code> for a member already paid
17	<code>placeBid</code> reverts <code>BiddingClosed</code> after the window
18	<code>withdrawBid</code> by the leader correctly recomputes the next-highest leader
19	A member may raise their own leading bid
20	Final round has exactly one eligible bidder and settles at zero discount

12.3 Unit — contribution and settlement

#	Test
21	<code>contribute</code> before due is on time; after due but pre-settlement is late
22	<code>contribute</code> consumes <code>pendingRewards</code> before pulling from the wallet
23	<code>contribute</code> reverts <code>AlreadyContributed</code> on a second call
24	A paid-out member must still contribute and can
25	<code>settleRound</code> with all contributions pays exactly <code>grossPot - skim - discount - topUp</code>
26	<code>settleRound</code> with one non-payer slashes exactly <code>C</code> from their collateral
27	<code>settleRound</code> with a non-payer whose collateral is exhausted marks them <code>Defaulted</code> and draws insurance
28	<code>settleRound</code> with no bids assigns the lowest- <code>joinIndex</code> unpaid member at zero discount
29	<code>settleRound</code> reverts <code>RoundNotSettleable</code> before <code>contributionDueAt</code>
30	<code>settleRound</code> reverts <code>RoundAlreadySettled</code> on a second call in the same block

31	Redistribution reaches every waiting member and excludes the winner and paid-out members
32	Redistribution dust goes to the insurance pool; nothing is stranded
33	<code>quotePayout(r, bps)</code> equals the payout <code>settleRound</code> actually produces — fuzzed over bps
34	<code>quoteTopUp(r, who)</code> equals the top-up actually applied — fuzzed over round index
35	Top-up is capped at <code>net</code> and does not revert when it would exceed the payout
36	Final round settlement transitions to <code>Completed</code> and finalizes reputation
37	Uncovered shortfall transitions to <code>Unwinding</code> and stops further rounds

12.4 Unit — withdrawals

#	Test
38	<code>claimPayout</code> reverts <code>PayoutNotOwed</code> for a non-winner
39	<code>claimPayout</code> zeroes the balance before transferring (reentrancy probe with a malicious token)
40	<code>withdrawRewards</code> is callable mid-circle
41	<code>withdrawCollateral</code> reverts <code>CollateralLocked</code> while <code>Active</code>
42	<code>withdrawCollateral</code> after <code>Completed</code> returns <code>posted - slashed</code>
43	<code>withdrawCollateral</code> during <code>Unwinding</code> distributes pro rata and never exceeds the balance

12.5 Unit — reputation and insurance

#	Test
44	Score movements match the published constants exactly, for each event type
45	Score saturates at 0 and at 1000 without wrapping
46	Tier thresholds are correct at each boundary
47	<code>Transfer</code> , <code>approve</code> , and <code>setApprovalForAll</code> all revert <code>Soulbound</code>
48	<code>locked(tokenId)</code> returns true; <code>tokenURI</code> returns a valid on-chain data URI
49	Only a factory-registered circle can write reputation
50	Pool <code>claim</code> returns a partial amount rather than reverting when under-reserved
51	Pool per-circle cap limits a single circle's cumulative draws
52	Pool has no code path that lets the owner withdraw reserves

12.6 Integration — full circle simulation

#	Test
53	Happy path. 8 members, 8 rounds, everyone pays on time, varied discounts. Assert every member paid exactly <code>8C</code> , received exactly one pot, and got their collateral back in full. Assert conservation (INV-6) at every round boundary.
54	One defaulter mid-circle. Member 5 stops paying at round 3. Assert collateral covers rounds 3-4, insurance covers 5-7, every other member is made whole, and the defaulter's record shows a default.
55	Early-payout walkaway. Member wins round 0 at max discount, then never pays again. Assert the top-up plus insurance leaves the

remaining members whole, and quantify the residual loss — this is the number to put in the README.

56	Cascade. Three members default in the same round. Assert graceful <code>Unwinding</code> and pro-rata collateral return with no revert and no stranded funds.
57	Bidding war. Ten bids in one window, several withdrawals, leader recomputation correct throughout.
58	Backend signature compatibility. A quote digest produced by the backend's EIP-712 encoding is accepted on-chain (§9.2).

12.7 Invariant tests

Handler-based, with actors that randomly join, bid, contribute, skip contributions, settle, and withdraw. Assert INV-1 through INV-8 (§11.2) after every call.

The handler should include a **malicious actor** that attempts reentrancy on every callback surface, and a **lazy actor** that never contributes — most real bugs surface from the interaction between those two and an honest majority.

12.8 Gas benchmarks

Record and keep in the repo. `settleRound` at `memberTarget = 20` is the one that matters; if it approaches anything uncomfortable, that is the signal to split `_finalizeReputation` into batches (§7.13).

Function	Target
<code>join</code>	< 200k
<code>contribute</code>	< 120k
<code>placeBid</code>	< 60k
<code>settleRound</code> (8 members, all paid)	< 400k
<code>settleRound</code> (20 members, 3 defaulters)	< 900k

13. Deployment

13.1 Order

Circular dependencies between the factory, reputation, and pool are resolved by deploying the factory last and passing it in afterwards via one-time setters.

```
1. MockUSDC                (testnet only)
2. WhamCircle implementation (never initialized; lock it with _disableInitializers())
3. WhamReputation           (factory address set post-deploy)
4. WhamInsurancePool         (factory address set post-deploy)
5. WhamCircleFactory(implementation, reputation, insurancePool)
6. reputation.setFactory(factory)    - one-time, then permanently immutable
7. insurancePool.setFactory(factory) - one-time, then permanently immutable
8. factory.setAllowedToken(usdc, true)
9. factory.setDefaultAgent(agentAddress)
10. insurancePool.fund(seedAmount)   - see §13.3
11. Verify all contracts on Arbiscan
```

Step 2 matters. An uninitialized implementation contract behind clones can be initialized by anyone. Call `_disableInitializers()` in the implementation's constructor. It is one line and its absence is a finding in every audit.

13.2 Script

```
// script/Deploy.s.sol
contract Deploy is Script {
    function run() external {
        uint256 pk = vm.envUint("DEPLOYER_PK");
        address agent = vm.envAddress("AGENT_ADDRESS");
        vm.startBroadcast(pk);

        MockUSDC usdc = new MockUSDC();
        WhamCircle impl = new WhamCircle();
        WhamReputation rep = new WhamReputation();
        WhamInsurancePool pool = new WhamInsurancePool(IERC20(address(usdc)));
        WhamCircleFactory factory = new WhamCircleFactory(address(impl), rep, pool);

        rep.setFactory(address(factory));
        pool.setFactory(address(factory));
        factory.setAllowedToken(address(usdc), true);
        factory.setDefaultAgent(agent);

        usdc.mint(msg.sender, 500_000e6);
        usdc.approve(address(pool), 50_000e6);
        pool.fund(50_000e6);

        vm.stopBroadcast();
        _writeDeploymentJson(...); // consumed by frontend and backend config
    }
}
```

`_writeDeploymentJson` must emit a file matching the shape the frontend expects in `CONTRACTS[chainId]` (Frontend §2.4) and the backend expects in `Wham:Contracts` (Backend §2.4). Generating one file both consume removes an entire category of “wrong address on the wrong environment” bugs.

13.3 Demo seeding

The demo needs a chain that has already lived a little (Frontend §11.1, Backend §14.1). Write this as a script, because you will run it more than once.

```
script/SeedDemo.s.sol
├─ Fund 10 wallets with mock USDC and ETH
├─ Deploy "Diaspora Circle #3": 8 members, 50 USDC, 14-day rounds
│   └─ All 8 join with varied agent quotes (75, 100, 100, 100, 125, 150, 100, 100 USDC)
│       └─ → produces visibly different multipliers on the agent page
│   └─ Rounds 0–2: settle with real bids (11%, 6%, 4% discounts)
│       └─ Round 2: member 5 misses → collateral drawn, marked delinquent
│           └─ Round 3: leave open in bidding with two live competing bids
├─ Deploy "Steady Six": 6 members, run to completion → reputation attestations minted
├─ Deploy "Weekly Ten": force a default that exhausts collateral → one real insurance claim
└─ Deploy "Ten Month Ten": forming, 4/10 filled, joinable by a judge
```

Use `vm.warp` between rounds. The seeded chain should let a judge land on `/circles`, join a real circle, place a real bid, and see a real agent decision — without any of it being a mock.

Seed the insurance pool before forcing the default in “Weekly Ten”, or the claim will pay zero and the pool page will show an empty state where the demo needs its one real claim.

14. Build order

Three weeks, in parallel with the frontend and backend builds.

Week 1 — Skeleton and the join path

#	Task
---	------

1	Foundry scaffold, <code>foundry.toml</code> with <code>via_ir</code> , OZ 5.x, CI running tests on every push
2	Types, enums, errors, events (\$4) — publish the ABI to the frontend and backend on day one , even with unimplemented function bodies, so all three teams generate against the same interface
3	<code>WhamReputation</code> with soulbound enforcement + on-chain <code>tokenURI</code> (\$8)
4	<code>WhamInsurancePool</code> (\$10)
5	<code>WhamCircleFactory</code> + clone deployment + <code>predictAddress</code> (\$5)
6	<code>WhamCircle.initialize</code> + storage + all view helpers (\$6)
7	EIP-712 quote verification + <code>join</code> + the backend-compatibility test (\$9.2)
8	Tests 1–12

Week 2 — The mechanism

#	Task
9	<code>placeBid</code> / <code>withdrawBid</code> + leader recomputation (\$7.5–7.6)
10	<code>contribute</code> with rewards netting (\$7.7)
11	<code>settleRound</code> — the whole of \$7.8, in the specified order
12	Payout-contingent top-up (\$2.5) + <code>quoteTopUp</code> matching settlement exactly
13	<code>claimPayout</code> , <code>withdrawRewards</code> , <code>withdrawCollateral</code> (\$7.9–7.11)
14	Tests 13–43
15	Deploy to Arbitrum Sepolia, verify, hand addresses to the frontend and backend

Week 3 — Hardening and the demo

#	Task
16	<code>Unwinding</code> path + pro-rata withdrawal + cascade handling
17	<code>try/catch</code> around reputation calls (\$11.1)
18	Invariant test suite (\$12.7) at depth
19	Integration scenarios 53–58
20	Gas benchmarks; split <code>_finalizeReputation</code> if needed
21	<code>SeedDemo.s.sol</code> producing the full demo chain state (\$13.3)
22	README documenting the mechanism, the non-goals (\$11.3), and the <code>decisionRef</code> verification chain (\$9.3)
23	<code>releaseExcessCollateral</code> — only if everything above is green

15. Worked end-to-end scenario

Use this as the reference for integration test 53 and as the narrative for the submission. `N = 8`, `C = 50 USDC`, `insuranceSkimBps = 150`, `maxDiscountBps = 1500`, `payoutCollateralBps = 5000`.

Setup. Eight members join with agent-quoted collateral between 75 and 150 USDC (bands A through D). Total collateral held: 850 USDC. Gross pot per round: 400 USDC.

Round 0. Mina bids 11% — she needs the money now. No one outbids her.

Gross pot	400.00	
- Insurance skim (1.5%)	6.00	→ pool
- Discount (11%)	44.00	
Net payout	350.00	
- Collateral top-up	100.00	→ her deposit rises from 75 to 175
Mina receives	250.00	

Redistribution: $44.00 \div 7 \text{ waiting} = 6.28 \text{ each}$, dust 0.04 → pool

Mina walked away with 250 of a 400 pot. In exchange, seven other members each earned 6.28 for waiting, the pool gained 6.04, and Mina's remaining 350 USDC obligation is now 50% collateralised. Every one of those consequences is visible in the UI before she bids.

Round 2. Reza wins at 4%. He then misses round 3.

Settlement of round 3:		
7 contributions received	350.00	
Reza's collateral drawn	50.00	(of 150 posted → 100 remaining)
Effective pot	400.00	
Shortfall	0.00	
Reza → Delinquent, reputation -60		

The other seven members do not notice. That is the entire point of the collateral.

Round 7 (final). One eligible member remains, so no auction. Settles at zero discount, 394.00 net. Circle completes.

Settlement. Each member has paid exactly 400.00 and received exactly one pot. Members who waited collected redistribution along the way; members who took early pots paid for it. Collateral returns in full to everyone except Reza, who gets back 100 of 150. His record permanently shows one missed round.

The number worth putting in the README: across the whole circle, no member who honoured their obligations lost a single unit, and the mechanism that guaranteed it was not trust.